1. What am I making?

RADIX ProofPath helps a candidate understand job readiness. Five teammates own five parts: JD Analytics -> Resume Parsing -> Profile Builder -> Talent Check -> Skill Matching. You own part 2.

Read a CV and show what supports each claim.

Read a candidate's CV and turn its skills, education, projects and experience into editable information. A CV mention is a claim, not a measured ability score.

Why your part matters

Typing every CV field is slow. Your part gives Profile Builder a useful starting point while letting the candidate correct mistakes.

What the user gives you

A text-based resume PDF or Word (.docx) file, plus optional permission to use Groq.

What you return

A reviewed resume record containing skill claims, source quotes and the background fields that can be supported by the CV.

A small example

Asha's synthetic CV says Python and describes a Python quiz. Show Python/COD and the quoted text, plus the project and education. Do not turn that mention into Python level 8.

Start with the working code

This is a working starting point, not an instruction to rebuild an app. Clone the WHOLE repository so shared support is available. Work mainly in resume_parsing/. Read this one guide, run it, then improve a problem you can demonstrate.

Your folder has this guide as PDF and Markdown, backend/, frontend/, tests/, samples/ and run.ps1. Shared support is stored once at the root. The Markdown contains the same teaching content and full examples as this PDF.

2. What exactly must I build?

Organizer requirements - keep these working

- Accept a resume PDF or DOCX, including layouts that differ from the sample.
- Use the LLM extraction pattern to find technologies and technical signal; preserve the existing Groq implementation. Rules-only output is a fallback, not verified LLM extraction.
- Return skills in the shared format, each with one RADIX category or OTHER and supporting evidence. Return education, projects and experience when reliably present.
- Run independently on at least two different resumes; send the reviewed output to Module 3. Test the missing official sample pack when it becomes available.

Already implemented

PDF/Word upload, section reading, direct Groq structured proposals, checked quotes, editable fields, negation/review flags and reviewed save/reload already work. No numeric proficiency is inferred. Actual Groq quality remains unverified.

Your development checklist

- 1. Run the existing upload screen and the supplied single-skill CV first. Read the source and compare every returned field.
- 2. Try a second layout. Check whether projects and internships stay in the right sections; missing fields should remain empty, not invented.
- 3. Reproduce and improve one real omission or layout problem. Add a regression test so that exact error cannot quietly return.
- 4. Save the reviewed CV, import it into Profile Builder and confirm that all twelve ratings are still Unknown. Test Groq separately if a private authorized key becomes available.

Currently missing or unverified

Original organizer resumes and live Groq evaluation are pending. OCR for image-only PDFs and perfect understanding of every layout are not implemented. Current rules can miss Rust, implicit data cleaning and some wrapped skills.

Extra ProofPath features

ProofPath connects a skill claim to the page and words that support it. Conflicting or negative claims remain visible for review. Preserve this trail; a reviewed claim is still not independently verified competence.

3. How does it work?

1. Upload	**2. Read sections**	**3. Extract fields**
4. Check quotes	**5. Review**	**6. Import**

- 1. Upload: The candidate chooses a PDF or Word CV. The existing reader handles text-based files and checks file validity.
- 2. Read sections: Page text and headings help separate education, projects, work and other background.
- 3. Extract fields: Groq identifies structured claims when configured and explicitly allowed. Otherwise local rules run with an honest label.
- 4. Check quotes: The server checks proposed words against the source. Unsupported fields cannot become trusted source facts.
- 5. Review: The candidate fixes names, fields, categories and claims, then saves a reviewed resume.
- 6. Import: Module 3 imports the reviewed record as a draft. Only a profile save makes it the profile used by assessments.

Words you will see

Backend = Python server code. Frontend = React browser screen. API = an address the screen calls to send/receive data. A contract/schema is the agreed field list, defined in shared/contracts/models.py. An LLM is a language model that interprets text; Groq runs it. Supabase is the cloud database service.

The shared RADIX categories

COD Coding; DSA Data Structures & Algorithms; OOD Object-Oriented Design; APTI Aptitude; COMM Communication; AI Artificial Intelligence; CLOUD Cloud; SQL SQL; SWE Software Engineering; SYSD System Design; NETW Networking; OS Operating Systems.

Each skill has exactly one category, or OTHER. Descriptive skill families are extra UI filters, not new rated categories. Unknown competency is null, never zero. Confidence describes extraction support, not ability. What-if changes never save a real profile.

4. Inputs and outputs

The candidate provides the CV; Module 3 receives the reviewed Extraction result.

POST /api/resumes/extract

This complete request example describes the upload form. Send the actual file, not this JSON object, to the endpoint. multipart/form-data means the browser form contains file bytes as well as ordinary fields.

Complete input example: resume_parsing/samples/input.json

```
{
  "method": "POST",
  "path": "/api/resumes/extract",
  "encoding": "multipart/form-data",
  "file": "practice-cv-SYNTHETIC.pdf",
  "hosted_consent": false
}
```

Important fields

- file / hosted_consent: File is required. Consent defaults to false and applies to this input only.
- name / email / section arrays: Background values supported by the CV. Empty arrays mean no reliable extracted entry; they are not proof of no experience.
- skills / evidence / source_document_id: Named claims, exact quotes and a private document link. verification_status stays unverified; confidence is separate from ability.
- id / revision / review_status: Keep the returned ID and version when saving. Set reviewed only after checking. Source and skill IDs in the printed example belong to that captured run.

GET /api/resumes/{id} loads the record. PATCH /api/resumes/{id} sends the complete edited Extraction with the same id/current revision and review_status=reviewed. Module 3 uses POST /api/profiles/import-resume with resume_id, a profile draft and optional replace_existing=false. Unreviewed imports are rejected.

The next page(s) show the COMPLETE captured response from the existing API, including defaults and empty fields. Long lines wrap for reading. Use samples/output.json for exact machine-readable JSON. IDs, receipt IDs, source hashes and times can differ between runs; do not hardcode them. JSON null means unknown/not present; [] means an empty list.

Complete output example

resume_parsing/samples/output.json


```
{
  "schema_version": "1.0",
  "id": "ad46106d-5cc7-496a-9d08-80c6cd58e2e3",
  "revision": 1,
  "source_type": "resume",
  "source_file": "practice-cv-SYNTHETIC.pdf",
  "company": null,
  "role": null,
  "skills": [
    {"skill_id": "a6ce5f95-27f9-43d3-945d-56e537022803", "skill_name": "Python",
  "category_code": "COD", "skill_family": "Programming Languages", "evidence": "Python",
  "confidence": "medium", "source_page": 1, "source_file": "practice-cv-SYNTHETIC.pdf",
  "source_document_id": "6a7af17a-635d-4efd-9587-f2cef80f28df", "source_type":
  "document", "evidence_type": "project_mention", "verification_status": "unverified",
  "claim_status": "claimed", "claim_basis": "explicit", "normalized_name": null,
  "source_start": null, "source_end": null}
  ],
  "requirement_groups": [],
  "name": "Asha Sample (synthetic)",
  "email": "asha@example.test",
  "education": ["Fictional Computing College - invented course."],
  "projects": ["Built a small Python quiz for invented training exercises."],
  "experience": [],
  "certifications": [],
  "internships": [],
  "hackathons": [],
  "achievements": [],
  "sections": [
    {"page": 1, "text": "SYNTHETIC TEACHING CV - NOT A REAL PERSON\n\n\nName: Asha Sample
   (synthetic)\n\nEmail: asha@example.test\n\nTechnical
  Skills\n\nPython\n\nProjects\n\nBuilt a small Python quiz for invented training
  exercises.\n\nEducation\n\nFictional Computing College - invented course."}
  ],
  "review_status": "needs_review",
  "extraction_mode": "local_limited",
  "parser_version": "local-layout-review-1.3",
  "input_method": "file",
  "model_version": null,
  "prompt_version": null,
  "llm_status": "rules_only",
  "llm_metadata": {},
  "warnings": ["Limited local extraction. Review name, section boundaries and skills; no
  ability levels inferred."],
  "document_id": "6a7af17a-635d-4efd-9587-f2cef80f28df"
}
```

5. How do I work on the code?

Install and run - Windows PowerShell

Install Python 3.10+, Node.js 24 LTS (includes npm), Git and an editor such as VS Code. Accept the installers' PATH option, reopen PowerShell and clone the full GitHub repository. Open a terminal at the repository root - the folder containing package.json.

```
.\resume_parsing\run.ps1 setup
# Terminal 1:
.\resume_parsing\run.ps1 api
# A SECOND terminal, same root:
.\resume_parsing\run.ps1 web
```

Open http://127.0.0.1:5173/. Server API help is at http://127.0.0.1:8000/docs. setup creates .venv and installs locked Python/npm dependencies plus Chromium for browser tests. Run setup once; no key is needed. Keep api and web running; Ctrl+C stops your own server. Only one module uses 8000/5173 at a time.

Open these files first

- resume_parsing/backend/service.py - Open first. Reads headings/background and supplies the resume extraction instructions.
- resume_parsing/backend/router.py - Connects the feature to common upload, load and review routes.
- resume_parsing/frontend/ResumeParsingPage.tsx - Displays upload, editable extracted sections, skill claims and review/save controls.
- resume_parsing/tests/test_extraction.py - Small backend examples that show how the parser is expected to behave.

shared/integrations/llm.py (Groq), shared/integrations/extraction.py (quotes and skills), shared/integrations/documents.py (reading files), shared/contracts/models.py (fields). Request these exact files from your assistant if the bug crosses the feature boundary.

Make one useful change

Read a failing example, locate the responsible function, change that part and add a test in your tests/ or frontend test file. Do not rename shared fields, duplicate common code or rewrite other modules. Ask the Scrum Master before changing shared contracts, storage or provider code.

What should I see?

Expect a CV file choice, extracted name/background, editable skills and source evidence, then Save reviewed resume. Show a project quote and an error state as well as the success screen.

Load synthetic inputs supplies pre-reviewed Mira records for independent operation. It is not fresh extraction. The tiny Asha terminal examples and the richer Mira browser examples are intentionally different inputs.

6. How do I know it works?

```
.\resume_parsing\run.ps1 example
.\resume_parsing\run.ps1 test
```

example runs the small teaching case in a fresh temporary database and checks its result. test runs your real standalone API check, Python tests, React tests, type checking, lint, your production build and your browser test. It stops on failure. Both use synthetic data; tests force rules mode and do not prove live LLM quality.

- Basic CV: Upload samples/practice-cv-SYNTHETIC.pdf, then .docx. Expect Asha Sample, Python/COD, the quiz project and invented education. Wrong: invented employment or an automatic rating.
- Second layout: Use a different synthetic CV in data/synthetic/reconstructed/ or extraction-eval/documents/. Compare sections and quotes, not candidate names or a fixed expected skill count.
- Negation/conflict: Try authorized text saying No production Kubernetes experience. It must not become a positive proven ability. Keep uncertainty visible for human review.
- Broken/empty CV: Upload an empty file or text renamed .pdf. Expect a clear error and the ability to try again. An image-only PDF needs readable text/OCR outside this implementation.
- Review/import: Save corrected fields, reload and import the reviewed result into Module 3. The fields should survive and all twelve ability ratings should stay Unknown.

Use an unfamiliar document too

Use only documents you own or have permission to process. Before running extraction, list expected skills and supporting words yourself. Compare missing and invented skills, sections, groups and quotes. A successful API response or valid JSON is not proof of correct extraction. Do not commit real CVs or send them to Groq without explicit permission.

If something fails

Read the FIRST error. Missing Python/npm: install and reopen the terminal. Missing packages: rerun setup. Browser cannot reach API: keep api running. Port busy: stop your own earlier server or coordinate a free port; do not kill unknown processes. Wrong extraction: compare the source, review and add a regression case.

7. Missing resources and what to do

What you can do now

All local teaching inputs are synthetic. No organizer pack, API key or cloud account is needed to run, edit and test your module. The tiny samples teach fields and behavior; the larger reconstructed pack remains under data/synthetic/. Neither is official organizer data.

Your samples - every file has a purpose

- expected.json - Small expected-result checklist; compare semantics, not generated IDs.
- input.json - Complete upload description or JSON request shown in this guide.
- output.json - Complete response captured from that actual API request.
- practice-cv-SYNTHETIC.docx - The same fictional CV in Word format.
- practice-cv-SYNTHETIC.pdf - Tiny fictional Asha CV for upload testing.

Groq: needed for live LLM verification, not offline practice

Use an authorized Groq account/key. If root .env does not exist, copy .env.example to .env; otherwise edit the existing private file carefully. Add these server-side values, restart api, choose hosted consent for ONE synthetic upload and check llm_status=processed plus source correctness. No LangChain wrapper is used; this implementation calls Groq directly.

```
RADIX_LLM_MODE=auto
RADIX_LLM_API_KEY=YOUR_PRIVATE_GROQ_KEY
RADIX_LLM_MODEL=openai/gpt-oss-20b
```

The placeholder is not a real key. Never put the key in frontend code, VITE_ variables, Git, a screenshot or an AI upload. If unavailable, continue rules-mode development and report live verification pending. If the model/key is unavailable, the configured fallback may return rules with llm_status=unavailable; do not call it model success.

8. Submit and explain your module

Completion checklist

[ ] I can run/explain it. [ ] Required inputs/outputs work. [ ] Normal/failure cases checked. [ ] Tests passed or failures reported. [ ] Contracts/source links preserved. [ ] No secrets/private CVs committed.

Push to YOUR own GitHub

Create an EMPTY private repository on your own account. From your full working clone, replace the URL below with YOUR new repository, review/stage only your changes, then run:

```
git remote rename origin scrum-master
git remote add origin https://github.com/YOU/YOUR-REPO.git
git add resume_parsing
.\.venv\Scripts\python.exe scripts/scan_handoff.py --staged
git commit -m "Verify my RADIX module"
git push -u origin main
```

Keep the whole working clone, including your source/tests/samples, guides, launcher and shared support. Never force-push. Grant the Scrum Master access if private.

Return URL, git rev-parse HEAD, changed files, actual test commands/results, three normal + three failure cases, synthetic screenshots and limitations. Distinguish implemented, mock-tested, live-verified and pending. The Scrum Master handles integration; you need not solve central merges.

A 45-second explanation for judges

My module reads a resume and prepares editable information for Profile Builder. It finds named skill claims and reliable education, projects and experience, with links to the supporting source. The existing Groq path checks structured proposals and quotes; local rules are a clearly labelled fallback. A person reviews the result before import. I test different layouts, missing fields and misleading claims. A CV mention never becomes a proficiency score. Live model quality and original organizer resumes remain separate checks.

Three likely questions

- Does Python in a CV prove expertise? No. It supports an unverified claim. Numeric self-ratings are entered separately.
- What if a field is missing? Leave it empty and let the candidate add it; do not fabricate details.
- What about scanned resumes? This reader needs text. Image-only PDFs need OCR, which is not included here.

Copy this prompt into your AI assistant

Help me complete module 2: Resume Parsing step by step using this guide. First run it; request the exact section-5 source files and tests before editing. Preserve working code and shared fields. Fix a reproduced issue and test it. Keep confidence separate from competency and scenarios separate from saved profiles. Never invent official data or request pasted keys. Help me submit my own GitHub URL, tested commit, results and honest limitations.